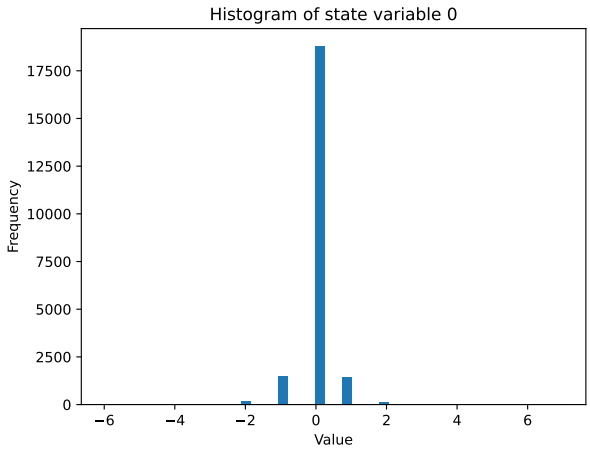
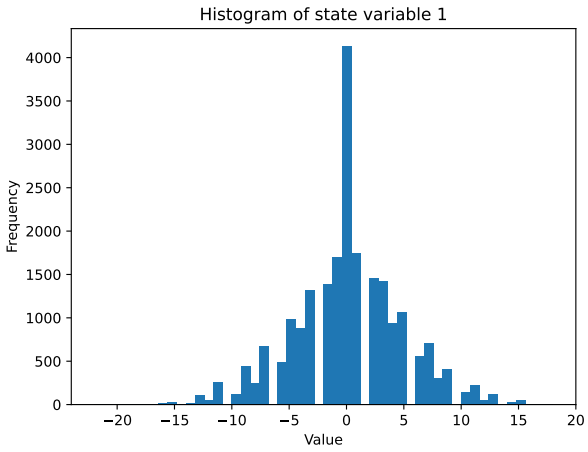
STA 5635 HW 12

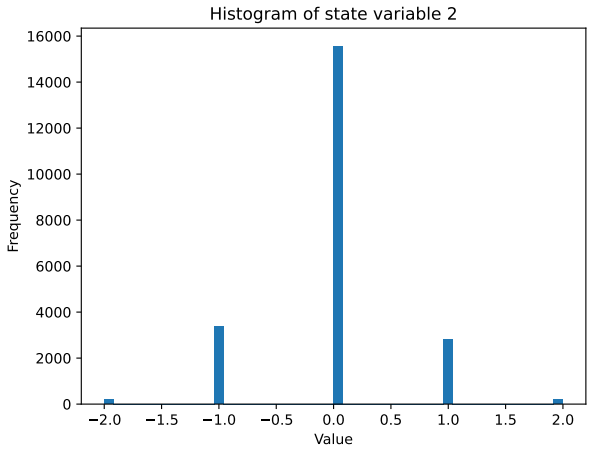
Palmer Swanson

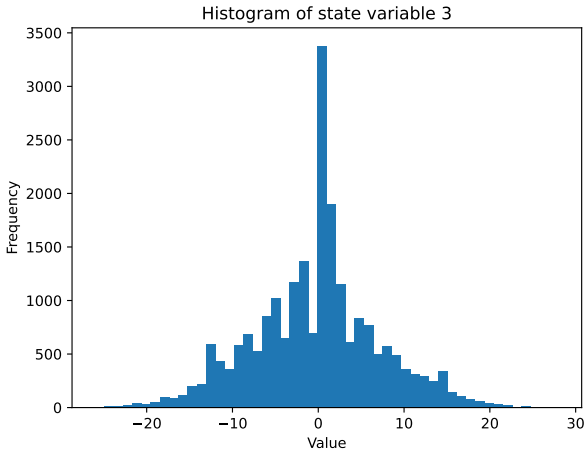
April 7, 2026

Part 1









State Variable	Min	Max	Num Discrete Values
0	-6	7	14
1	-22	18	41
2	-2	2	5
3	-26	28	55
Total			157850

Parts 2, 3, and 4

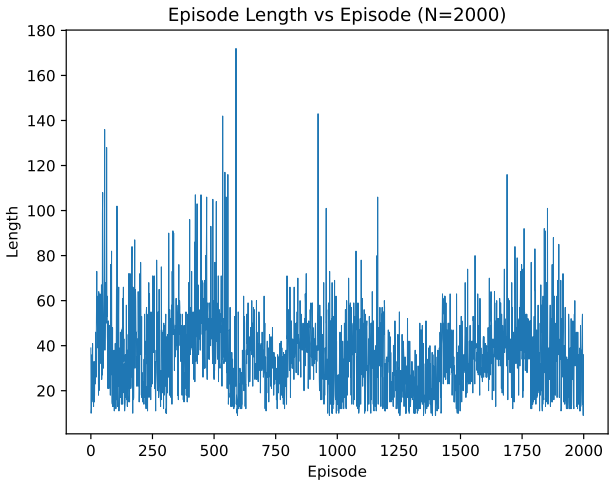


Figure 2: Episode Length vs Episode (N=2000)

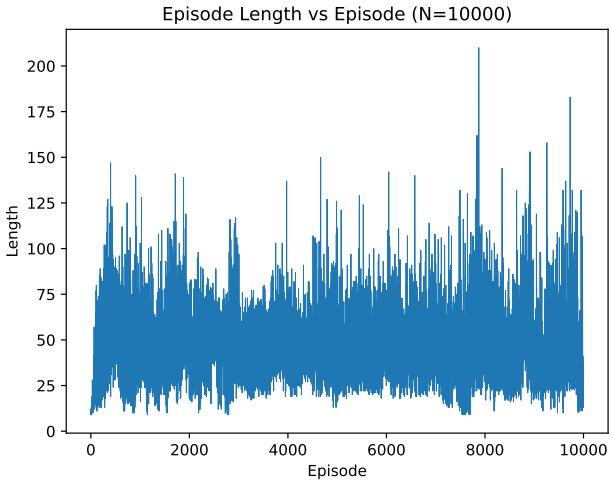


Figure 3: Episode Length vs Episode (N=10000)

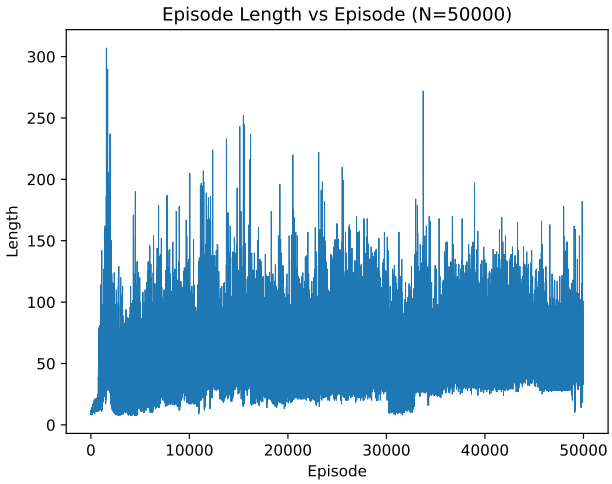


Figure 4: Episode Length vs Episode (N=50000)

N	% Nonzero (Random)	% Nonzero (Max)	Avg Ep. Length (Last 1000)
2000	1.307	1.714	33.756
10000	2.771	3.721	41.756
50000	5.565	6.987	53.694

Code

```
import gymnasium as gym
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random

#https://gymnasium.farama.org/introduction/basic_usage/
#I think the first three states are position, velocity, and angle.
#not sure what the fourth state is

#https://gymnasium.farama.org/environments/classic_control/cart_pole/
#ah.  fourth is angular velocity.
#need to create a cartpole environment
env = gym.make("CartPole-v1")

n_episodes = 1000

states = []

#part a
#loop over and collect the states of 1000 episodes
for i in range(n_episodes):

    #reset enviro at the start of each episode
    state, info = env.reset()
    done = False

    #run this until done
    while not done:
        #append to the states list
        states.append(state)

        #take a random action (0 or 1)  and observe the next state and reward
        a = random.randrange(2)
        observation, reward, terminated, truncated, info = env.step(a)

        #epsiode ends if one of these is true
        done = terminated or truncated

        #update state
        state = observation

env.close()

#need to multiply by 10 and converting to integers
```

```
states = (np.array(states)*10).astype(int)

#plot histograms and save
#usig 50 bins for each state variable, and save as pdfs in the figures folder
for i in range(4):
    plt.figure()
    plt.hist(states[:, i], bins=50)
    plt.title(f"Histogram␣of␣state␣variable␣{i}")
    plt.xlabel("Value")
    plt.ylabel("Frequency")
    plt.savefig(f"assignments/hw12/figures/hist_state_variable_{i}.pdf",
    dpi=400, bbox_inches="tight")
    plt.close()
    #plt.show()

#need to find the range of each of the state variables
#also need to report the number of discrete sate values for each state variable
mins = states.min(axis=0)
maxs = states.max(axis=0)
num_vals = maxs - mins + 1

#get total number of discrete states
num_total_states = np.prod(num_vals)
print(f"Total␣number␣of␣discrete␣states:␣{num_total_states}")

#build dataframe
df = pd.DataFrame({
    "State␣Variable": [0, 1, 2, 3],
    "Min": mins,
    "Max": maxs,
    "Num␣Discrete␣Values": num_vals
})

#add a total row
total_row = pd.DataFrame({
    "State␣Variable": ["Total"],
    "Min": [""],
    "Max": [""],
    "Num␣Discrete␣Values": [num_total_states]
})

df = pd.concat([df, total_row], ignore_index=True)

#export table as latex
latex_table = df.to_latex(index=False)

with open("assignments/hw12/output/state_summary_part1.tex", "w") as f:
```

```
    f.write(latex_table)



#parts 2, 3, and 4
#loop over these
N_values = [2000, 10000, 50000]
gamma = 0.9

results = []

for N in N_values:
    print(f"\n=====␣N␣=␣{N}␣=====")

    #initialize Q table
    Q = np.zeros((num_vals[0], num_vals[1], num_vals[2], num_vals[3], 2))

    for i in range(N):
        state, info = env.reset()
        done = False

        while not done:
            #convert state to discrete state and clip to the min and max values
            discrete_state = (state * 10).astype(int)
            discrete_state = np.clip(discrete_state, mins, maxs)
            s_idx = discrete_state - mins

            #take random action
            a = random.randrange(2)
            next_state, reward, terminated, truncated, info = env.step(a)
            done = terminated or truncated

            #convert next state to discrete state and clip to the min and max
    values
            disc_next = (next_state * 10).astype(int)
            disc_next = np.clip(disc_next, mins, maxs)
            s_next_idx = disc_next - mins

            #update Q table
            Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
                reward + gamma * np.max(
                    Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
                )
            )

            state = next_state

```

```
    #report percent nonzero after random phase
    percent1 = 100 * np.count_nonzero(Q) / Q.size

    eps = []

    for i in range(N):
        state, info = env.reset()
        done = False
        steps = 0

        #stop any episode with more than 2000 steps
        while not done and steps < 2000:
            #convert state to discrete state
            discrete_state = (state * 10).astype(int)
            discrete_state = np.clip(discrete_state, mins, maxs)
            s_idx = discrete_state - mins

            a = np.argmax(Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3]])

            next_state, reward, terminated, truncated, info = env.step(a)
            done = terminated or truncated

            #convert next state to discrete state
            disc_next = (next_state * 10).astype(int)
            disc_next = np.clip(disc_next, mins, maxs)
            s_next_idx = disc_next - mins

            #update Q table
            Q[s_idx[0], s_idx[1], s_idx[2], s_idx[3], a] = (
                reward + gamma * np.max(
                    Q[s_next_idx[0], s_next_idx[1], s_next_idx[2], s_next_idx[3]]
                )
            )

            #set state to next state and increment steps
            state = next_state
            steps += 1

        #memorize episode length
        eps.append(steps)

    #report percent non zero after max action phase
    percent2 = 100 * np.count_nonzero(Q) / Q.size
    avg_last_1000 = np.mean(eps[-1000:])

    #save objects we need to report in a table
    results.append({
```

```
        "N": N,
        "% Nonzero (Random)": percent1,
        "% Nonzero (Max)": percent2,
        "Avg Ep. Length (Last 1000)": avg_last_1000
    })

    # save plot
    plt.figure()
    plt.plot(eps, linewidth=0.3)
    plt.title(f"Episode Length vs Episode (N={N})")
    plt.xlabel("Episode")
    plt.ylabel("Length")
    plt.savefig(f"assignments/hw12/figures/episode_length_N{N}.pdf", dpi=300,
    bbox_inches="tight")
    plt.close()

#convert results to df and export
results_df = pd.DataFrame(results)

#need escapce for the % signs
latex_table = results_df.to_latex(
    index=False,
    escape=True,
    float_format="%.3f"
)

#save latex table
with open("assignments/hw12/output/results_summary.tex", "w") as f:
    f.write(latex_table)
```